

Universidad del Valle de Guatemala

Facultad de Ingeniería

Data Science

Catedrático: Luís Furlán



Proyecto 2.

Informe Final

Nelson Eduardo García Bravatti 22434

Joaquín André Puente Grajeda 22296

José Antonio Mérida Castejón 201105

Guatemala, noviembre de 2025

Planteamiento Inicial del Problema

1. Situación Problemática

Las enfermedades transmitidas por mosquitos (p. ej., dengue, zika y chikunguña) requieren sistemas de vigilancia oportunos. Los métodos tradicionales son costosos y lentos; por ello, proyectos de ciencia ciudadana como Mosquito Alert permiten que la población reporte observaciones y fotos de mosquitos, que posteriormente son validadas por especialistas y utilizadas para la gestión de salud pública. Automatizar la validación de imágenes reduciría la carga de trabajo y aceleraría las decisiones de control vectorial.

2. Problema Científico

¿Es posible desarrollar un modelo de visión por computadora que detecte y clasifique mosquitos vectores en imágenes capturadas por ciudadanos, reduciendo la carga de validación manual?

3. Objetivo General

Desarrollar un modelo de aprendizaje automático que detecte y clasifique mosquitos vectores en imágenes capturadas por ciudadanos.

4. Objetivos Específicos

- a. Realizar un análisis exploratorio del dataset de imágenes de mosquitos para identificar características relevantes y desafíos de los datos.
- b. Implementar técnicas de preprocesamiento de imágenes y estrategias para abordar desbalances de clases y variabilidad en calidad de imágenes.
- c. Entrenar y comparar diferentes arquitecturas de detección y clasificación de objetos para evaluar su desempeño en la identificación de mosquitos.
- d. Evaluar los modelos utilizando métricas apropiadas para tareas de detección y clasificación, seleccionando el modelo con mejor rendimiento.

- e. Documentar la solución desarrollada para garantizar reproducibilidad y facilitar su implementación.

Investigación Preliminar

Enfermedades Transmitidas por Vectores

A nivel global, las enfermedades transmitidas por vectores representan más del 17% de todas las enfermedades infecciosas y causan más de 700,000 muertes anuales (WHO, 2024). El dengue se ha convertido en la enfermedad viral transmitida por vectores de mayor propagación, afectando aproximadamente a 3,900 millones de personas en más de 132 países (WHO, 2024). La región de las Américas vivió la mayor epidemia de dengue registrada en 2024, con más de 13 millones de casos reportados, triplicando el récord del año anterior (PAHO, 2025).

Dataset del Mosquito Alert Challenge

El dataset contiene imágenes anotadas de seis clases que incluyen los principales vectores de arbovirosis y malaria: *Aedes aegypti*, *Aedes albopictus*, *Anopheles* (género), *Culex* (género), *Culiseta* (género) y el complejo *Aedes japonicus/koreicus*.

Especies y Géneros del Dataset

Aedes aegypti (especie)

Rasgos morfológicos distintivos: patrón en "lira" blanco característico en el tórax; cría principalmente en recipientes artificiales con agua estancada; actividad de picadura diurna con picos al amanecer y atardecer. Vectores principales: dengue, chikungunya, Zika y fiebre amarilla. Esta especie es responsable de la transmisión primaria del dengue, con casos que aumentaron de 26.45 millones a 58.96 millones entre 1990 y 2021 (Zhang et al., 2025).

Aedes albopictus (especie)

Conocido como "mosquito tigre asiático" por la franja blanca única longitudinal en el centro del tórax; tolera climas más fríos que *Ae. aegypti*, permitiendo su expansión a regiones templadas. Vectores: chikungunya, dengue y otros arbovirus; presenta menor antropofilia que *Ae. aegypti*, lo que puede influir en los patrones de transmisión.

Aedes japonicus/Aedes koreicus (complejo)

Mosquitos invasores de zonas templadas, morfológicamente similares, agrupados como complejo debido a la dificultad de diferenciación. Competencia vectorial demostrada en laboratorio para CHIKV/ZIKV/WNV, aunque su relevancia epidemiológica en campo permanece incierta.

Anopheles (género)

Características únicas: palpos tan largos como la probóscide y postura de reposo característica con el abdomen elevado; actividad crepuscular y nocturna. Importancia epidemiológica: únicos transmisores de malaria en humanos (solo hembras), causando aproximadamente 249 millones de casos y más de 608,000 muertes anuales (WHO, 2024).

Culex (género)

Comportamiento: actividad nocturna predominante; muchas especies prefieren alimentarse de aves (ornitofílicas). Vectores importantes de virus del Nilo Occidental (WNV) y encefalitis según especie y región; *Cx. pipiens*, *Cx. quinquefasciatus* y *Cx. tarsalis* son especies clave en la transmisión de WNV.

Culiseta (género)

Adaptaciones ecológicas: varias especies adaptadas a climas fríos; generalmente de mayor tamaño corporal. Importancia vectorial específica: *Culiseta melanura* mantiene el ciclo enzoótico de encefalitis equina del Este (EEE).

Impacto Económico y Salud Pública

Carga económica

Las especies invasoras *Aedes* (especialmente *aegypti* y *albopictus*) han acumulado costos reportados de al menos US\$ 94.7 mil millones durante 45 años (1975-2020), con costos que se han incrementado 14 veces desde los años 2000 (Roiz et al., 2024). Los gastos en daños y pérdidas superan en diez veces la inversión en prevención y manejo.

Impacto sanitario

En 2024, solo en las Américas se reportaron más de 13 millones de casos de dengue, con 22,684 casos severos (0.17%) y 8,186 muertes (tasa de letalidad 0.063%) (PAHO, 2025). La carga global del dengue se ha duplicado aproximadamente entre 1990 y 2021 (Zhang et al., 2025).

Análisis Inicial

Problema y Dataset

El problema planteado corresponde a una tarea híbrida de detección y clasificación de objetos en imágenes reales capturadas en condiciones no controladas. El Mosquito Alert Challenge Phase 1 proporciona un conjunto de imágenes reales anotadas con bounding boxes y etiquetas de clase,

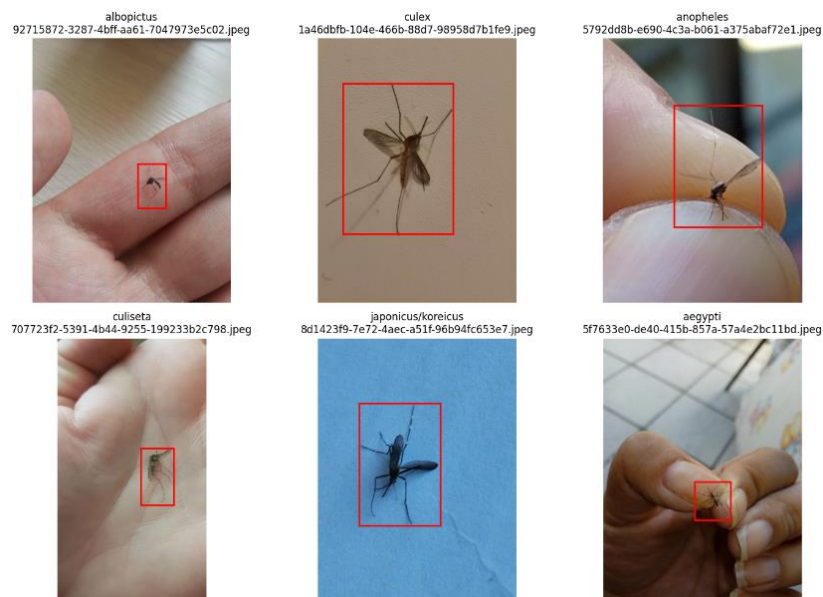
reflejando las condiciones reales de captura por parte de ciudadanos sin entrenamiento especializado. Las seis clases incluyen tanto especies específicas (*Aedes aegypti*, *A. albopictus*) como géneros completos (*Anopheles*, *Culex*, *Culiseta*) y un complejo de especies (*A. japonicus/koreicus*), lo que introduce variabilidad en el nivel taxonómico de clasificación.

Dificultades Previstas

Las imágenes capturadas por ciudadanos presentan condiciones heterogéneas de iluminación, enfoque, resolución y ángulos de captura, contrastando con datasets controlados de laboratorio. Además, es probable que exista una distribución desigual entre las clases, especialmente considerando que algunas especies son más comunes en áreas urbanas donde se concentra la participación ciudadana. Por último, las diferencias entre géneros relacionados pueden ser sutiles y requerir características específicas que pueden no ser claramente visibles en fotografías de campo.

Análisis Exploratorio

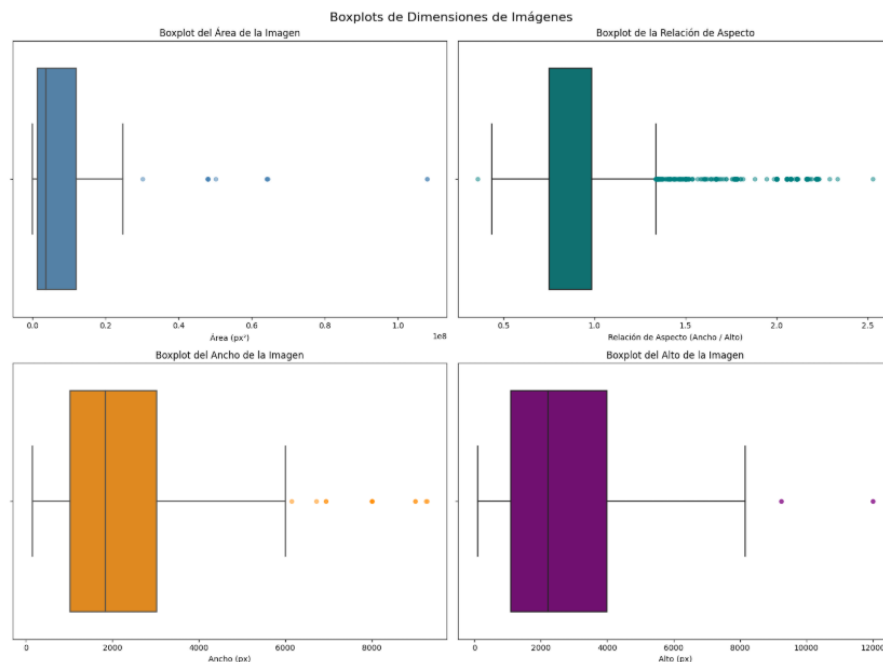
Como punto de partida del análisis exploratorio, se verificó el volumen de datos y su estructura de almacenamiento. El dataset consta de 8,205 imágenes en el conjunto de entrenamiento y 2,675 en el de prueba. No obstante, dado que las etiquetas (labels) del conjunto de prueba son privadas, es necesario utilizar exclusivamente el conjunto de entrenamiento para el desarrollo y evaluación de los modelos. La información disponible incluye las dimensiones de la imagen, las coordenadas del bounding box y la etiqueta de la especie.



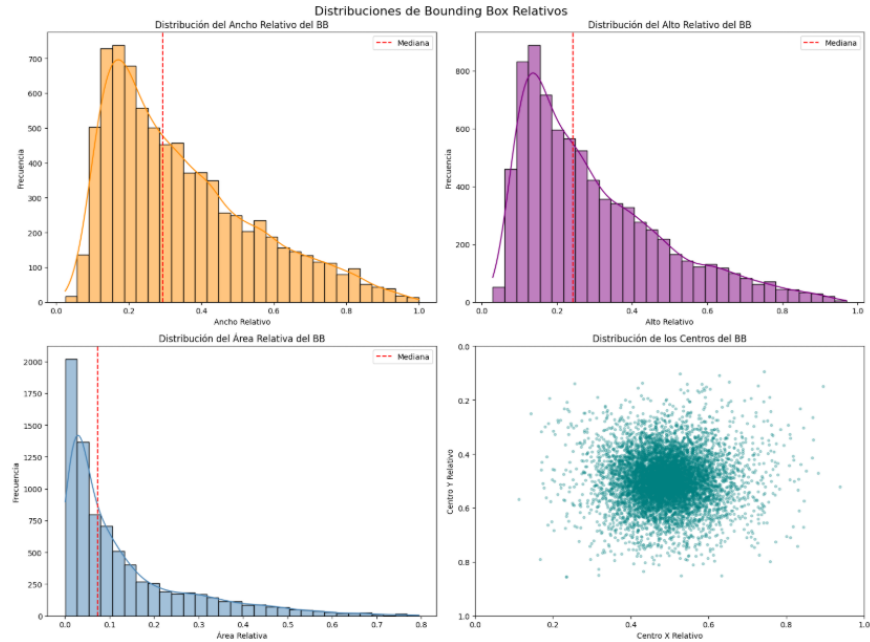
El primer desafío metodológico surgió al definir la estrategia de limpieza y exploración para un problema de visión por computadora. Al tratarse de un dominio nuevo para el equipo, se realizó una investigación previa para establecer criterios adecuados. Si bien se ejecutaron validaciones

inmediatas, como la identificación de bounding boxes fuera de rango y valores duplicados, también se incorporaron estándares de la industria. Estos incluyeron descartar entradas donde el objeto ocupara más del 80% de la imagen e identificar objetos irreconocibles por su tamaño (menores a 10px). Tras este proceso de depuración, el dataset final se redujo a 7,966 observaciones.

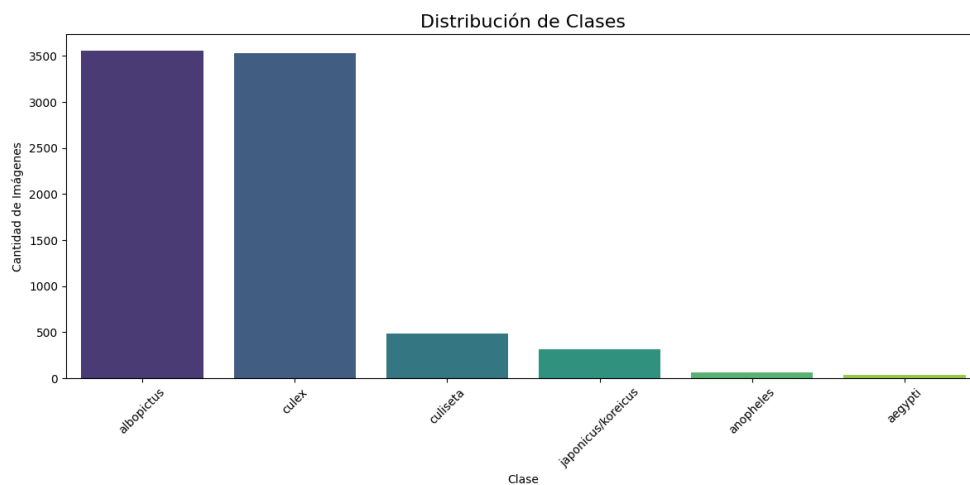
En lo cuanto, al análisis exploratorio, el enfoque se centró en las distribuciones del tamaño de las imágenes, el posicionamiento de los bounding boxes y la frecuencia de especies. Para complementar el estudio de las dimensiones, se generaron dos variables derivadas: área y relación de aspecto (aspect ratio). Se concluyó que existe una alta variabilidad en estos valores, lo cual fundamenta la necesidad de estandarizar las entradas antes de iniciar la fase de modelado.



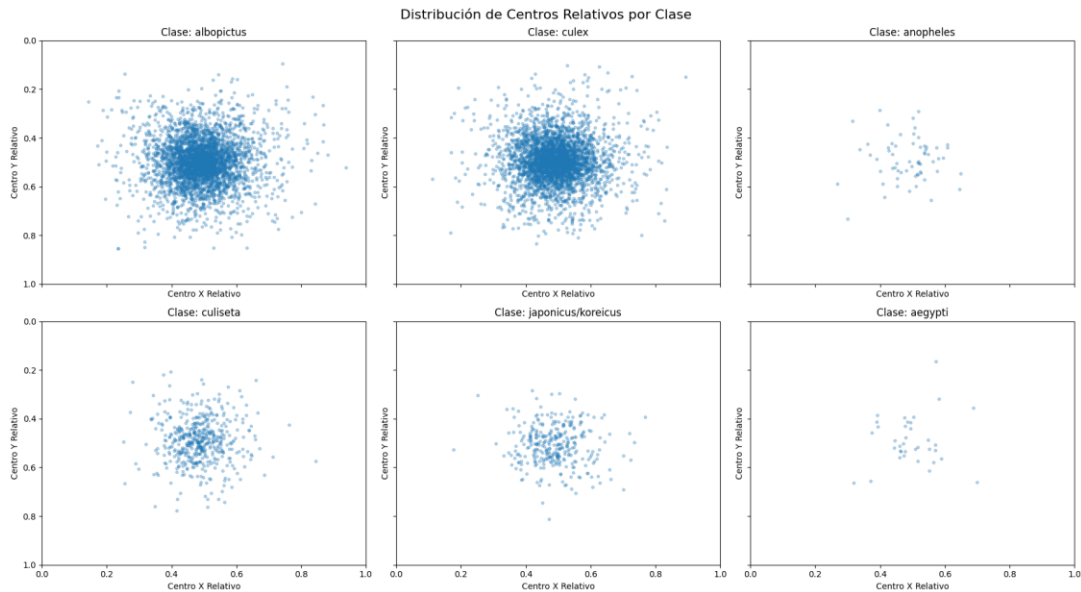
Posteriormente, se profundizó en el análisis de los bounding boxes mediante la creación de variables auxiliares sobre su posición y tamaño relativo. Los datos revelan que, en la mayoría de las observaciones, los mosquitos ocupan una proporción mínima de la imagen (menos del 15%) y tienden a ubicarse cerca del centro. Este hallazgo sugiere la utilidad de aplicar técnicas de data augmentation para introducir mayor variabilidad en posiciones y tamaños, mejorando así la capacidad de generalización de los modelos.



Durante esta etapa se identificó un segundo desafío significativo: el desbalance de clases. Al examinar la distribución de categorías, se observó que dos clases dominantes representan aproximadamente el 85% de los datos. Esta disparidad indica que será indispensable implementar estrategias de re-muestreo, como oversampling, durante el entrenamiento para evitar sesgos en el modelo.



Finalmente, se examinó la distribución de las variables numéricas en función de cada categoría. Los resultados evidencian los efectos del desbalance de clases mencionado anteriormente; específicamente, se nota que variables como el centro del bounding box carecen de diversidad representativa en las clases minoritarias.



Elección Inicial de Modelos

Para la elección inicial de modelos, realizamos una investigación sobre diferentes arquitecturas end-to-end (o 'all-in-one') capaces de realizar detección y clasificación simultánea. Al ser nuestro primer acercamiento a la visión por computadora, buscábamos familiarizarnos con estas tecnologías. Esta evaluación preliminar fue crucial para darnos cuenta de las limitaciones de precisión de estos modelos al intentar resolver ambas tareas conjuntamente. La investigación y selección de los modelos fue la siguiente:

YoloV8

Este modelo es la iteración más reciente de la familia YOLO y se clasifica como un detector de una sola etapa. Su principal ventaja es el equilibrio que logra entre velocidad y precisión, convirtiéndolo en una opción muy popular para aplicaciones en tiempo real. Además de la detección clásica, YOLOv8 es versátil porque soporta tareas de segmentación y estimación de pose. Su arquitectura es fácil de implementar gracias a una API accesible que permite exportar el modelo a formatos optimizados para producción (Ultralytics, 2023).

SSD

El SSD (Liu et al., 2016) fue clave en demostrar que los detectores podían ser eficientes realizando todo el proceso en una única pasada (single-shot), eliminando la necesidad de etapas complejas de propuesta de regiones. Se considera una arquitectura ligera, diseñada para mantener una buena capacidad de generalización sin demandar demasiados recursos de hardware. Por ello, suele ser la elección predilecta cuando se trabaja con CPUs o dispositivos móviles donde la potencia de cómputo es limitada.

Faster R-CNN

Considerado el estándar de los detectores de dos etapas, Faster R-CNN (Ren et al., 2015) prioriza la exactitud por encima de la velocidad. Funciona proponiendo regiones de interés en una primera fase y clasificándolas en una segunda, lo que le permite ser muy preciso localizando objetos pequeños o que están parcialmente ocultos. Aunque es computacionalmente más pesado que los modelos de una etapa, sigue siendo la referencia principal en investigación para comparar el desempeño de nuevas arquitecturas.

DetR

DETR (Carion et al., 2020) introdujo un enfoque diferente al utilizar Transformers para la detección, sustituyendo procesos manuales tradicionales (como los anchors) por un mecanismo de atención global. Esto permite que el modelo "entienda" mejor las relaciones espaciales entre los objetos de la imagen. Aunque el original era lento de entrenar, versiones modernas como RT-DETR han logrado optimizarlo para funcionar en tiempo real, siendo útil en escenarios donde la estructura de la escena es compleja

RetinaNet

RetinaNet (Lin et al., 2017) se diseñó para cerrar la brecha entre la velocidad de los detectores de una etapa y la precisión de los de dos etapas. Su gran aporte es la Focal Loss, una función de pérdida que ayuda al modelo a enfocarse en los ejemplos difíciles de clasificar, mitigando los problemas cuando hay un gran desbalance de clases (por ejemplo, mucho fondo vacío y pocos objetos). Es un excelente punto medio que ofrece robustez sin la complejidad de arquitecturas más pesadas.

EfficientDet

Este modelo se enfoca en la escalabilidad y la eficiencia. Utiliza una técnica de escala compuesta para optimizar la resolución, profundidad y ancho de la red simultáneamente, logrando un rendimiento alto con menos parámetros que sus competidores. EfficientDet (Tan et al., 2020) es ideal para despliegues en el borde (edge computing), como drones o móviles, donde el consumo de energía y la memoria son restricciones importantes.

Elección de Modelos

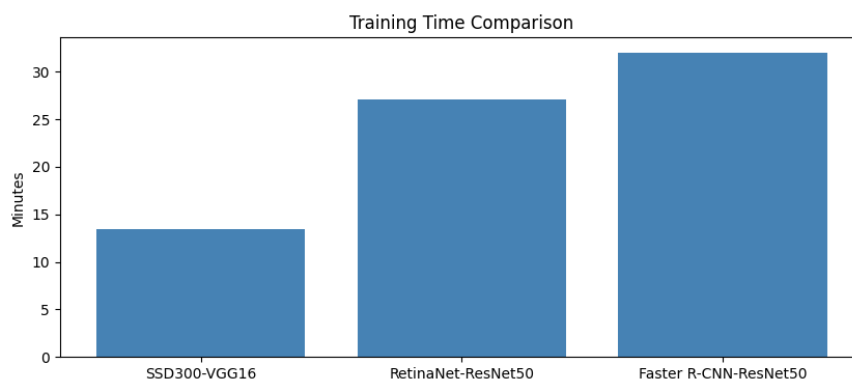
Para la fase inicial, decidimos trabajar con Faster R-CNN, RetinaNet y SSD porque representan tres enfoques complementarios que nos permiten evaluar distintas compensaciones (trade-offs). Seleccionamos Faster R-CNN como nuestra línea base de alta precisión y estabilidad. Incluimos RetinaNet para probar su eficacia manejando el desbalance de clases mediante la Focal Loss. Finalmente, elegimos SSD por su simplicidad y rapidez, lo que nos permite analizar una opción viable para entornos con recursos computacionales limitados.

Resultados Preliminares

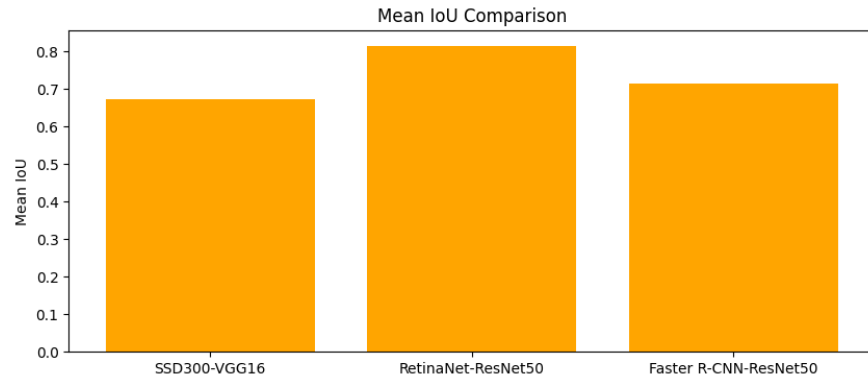
Metodología de Preprocesamiento y Entrenamiento

El flujo de trabajo para la preparación de los datos se implementó utilizando la librería Albumentations dentro de un ecosistema PyTorch. Las imágenes fueron redimensionadas a una resolución uniforme de 320 320 píxeles y normalizadas utilizando los estándares de ImageNet (media y desviación estándar). Para aumentar la robustez del modelo, se aplicaron técnicas de data augmentation al conjunto de entrenamiento, incluyendo volteo horizontal (horizontal flip, $p=0.5$) y ajustes aleatorios de brillo y contraste. La división del dataset se realizó mediante un muestreo estratificado (stratified split) para garantizar que la distribución de clases se mantuviera consistente entre los conjuntos de entrenamiento, validación y prueba. Adicionalmente, para mitigar el desbalance de clases identificado previamente, se implementó un WeightedRandomSampler. Este mecanismo asigna pesos inversos a la frecuencia de cada clase, forzando al modelo a prestar mayor atención a las categorías minoritarias durante los batches de entrenamiento.

En términos de costo computacional, se observa una correlación directa entre la complejidad de la arquitectura y el tiempo requerido. SSD300 fue el modelo más eficiente, completando el proceso en solo 13.4 minutos, lo cual es consistente con su diseño de una sola etapa (single-shot). Por el contrario, Faster R-CNN requirió 32.0 minutos (más del doble), un costo esperado debido a la sobrecarga computacional que implica su Red de Propuesta de Regiones (RPN). RetinaNet se ubicó en un punto intermedio (27.1 min), reflejando el peso adicional de la Feature Pyramid Network frente a SSD.



Al analizar qué tan bien se ajustan las "cajas" predichas a los objetos reales, RetinaNet obtuvo el promedio más alto con un Mean IoU de 0.814. Esto indica que, cuando el modelo logra detectar un objeto, el ajuste de las coordenadas es muy preciso. SSD obtuvo el valor más bajo (0.672), aunque este dato debe interpretarse con cautela debido a su bajo volumen de predicciones, lo que hace que su promedio sea estadísticamente menos representativo que el de Faster R-CNN (0.715), el cual mantuvo una consistencia sólida a lo largo de un gran número de detecciones.

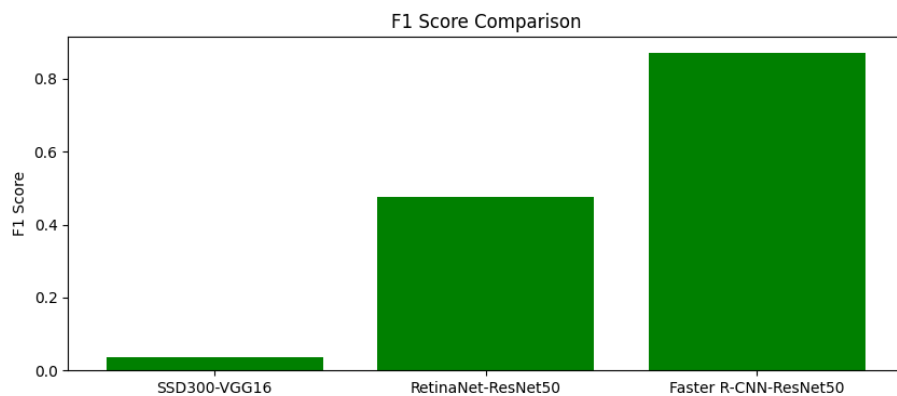


Como se ilustra en la gráfica comparativa de F1-Score, existe una disparidad crítica en la capacidad de los modelos para equilibrar precisión y exhaustividad (recall):

Faster R-CNN (0.871): Se consolida como el modelo superior. Su arquitectura de dos etapas logró recuperar el 83.8% de los objetos (Recall) manteniendo una precisión del 90.6%, demostrando ser la opción más robusta para este problema.

RetinaNet (0.476): A pesar de su buena localización (IoU), su desempeño global se vio penalizado por un Recall bajo (49.8%). El modelo falló en detectar aproximadamente la mitad de los objetos presentes en el conjunto de prueba.

SSD300 (0.037): El rendimiento de este modelo fue deficiente. Aunque su precisión (55.4%) no fue despreciable, el modelo falló estrepitosamente en Recall (1.9%). De los 1,600 objetos (ground-truths) en el set de prueba, SSD generó únicamente 56 predicciones. Esto sugiere que la resolución de entrada o la falta de capacidad de la red base (VGG16) impidieron que el modelo extrajera características suficientes para validar la presencia de objetos, comportándose de manera excesivamente conservadora.



Los resultados obtenidos demuestran que, si bien arquitecturas de dos etapas como Faster R-CNN ofrecen una ventaja significativa en este dominio, la estrategia de utilizar un único modelo para resolver simultáneamente detección y clasificación presenta rendimientos desiguales. La incapacidad de modelos ligeros como SSD para converger, sumada a las limitaciones de recall en RetinaNet, sugiere que abordar el problema desde una arquitectura monolítica impone

restricciones severas al intentar optimizar tareas tan dispares. Esta disparidad en el rendimiento motiva la exploración de una metodología modular: desacoplar el problema mediante un enfoque híbrido que utilice un detector dedicado exclusivamente a la localización de regiones, seguido de un clasificador especializado para la identificación de especies, permitiendo así una optimización independiente y más efectiva de cada etapa.

Rediseño de Modelos

Dadas las limitaciones observadas en la fase anterior, se decidió reorientar la estrategia hacia un enfoque modular, desacoplando las tareas de detección y clasificación. Para ello, se exploraron dos ecosistemas de desarrollo diferenciados: por un lado, la librería Ultralytics (YOLO), que ofrece una solución "todo en uno" caracterizada por su velocidad y facilidad de despliegue; y por otro, implementaciones personalizadas en PyTorch, que permiten el acceso a modelos del estado del arte (SOTA) con un mayor grado de control y flexibilidad.

Para la etapa de detección, la selección se centró en tres arquitecturas clave. Se retomaron Faster R-CNN y RetinaNet implementados en PyTorch, valorando su robustez y manejo del desbalance de clases respectivamente. A estos se sumó YOLOv8 en su versión de detección, introducido como una alternativa moderna diseñada para maximizar la eficiencia en inferencia sin sacrificar métricas de localización.

En cuanto a la fase de clasificación, se evaluaron tres modelos con paradigmas arquitectónicos distintos. Primero, YOLOv8-cls, para probar la consistencia de mantener un mismo framework en todo el pipeline. Segundo, ConvNeXt, una evolución de las redes convolucionales tradicionales que incorpora principios de diseño modernos. Y tercero, Vision Transformer (ViT), que aplica mecanismos de atención para capturar dependencias globales en la imagen, representando el enfoque más vanguardista pero computacionalmente exigente.

Entrenamiento de Modelos

La implementación de estos modelos supuso contrastar dos filosofías de desarrollo opuestas. Por un lado, el enfoque de YOLO ("Fast & AIO") permitió iteraciones rápidas gracias a su alto nivel de abstracción; su arquitectura interna gestiona automáticamente el preprocesamiento, las transformaciones de imágenes en tiempo real y la coordinación del bucle de entrenamiento (training loop), actuando casi como una "caja negra" optimizada. Por otro lado, la ruta de PyTorch ("Build it on your own") presentó una curva de aprendizaje más pronunciada, requiriendo la programación manual de dataloaders y la integración explícita de técnicas de aumento de datos.

En este entorno, se implementaron pipelines personalizados utilizando librerías como Albumentations, aplicando rotaciones, volteos horizontales y normalización estadística específica para enriquecer la variabilidad del dataset de entrada.

Sin embargo, esta "facilidad" de YOLO reveló limitaciones importantes al enfrentar problemas específicos de los datos, como el desbalance de clases. Al intentar entrenar YOLOv8-cls, la abstracción del framework dificultó la inyección de estrategias de re-muestreo personalizadas. Aquí es donde la flexibilidad de PyTorch resultó superior: aunque más compleja de configurar, permitió la implementación precisa de un WeightedRandomSampler para forzar al modelo a ver ejemplos de las clases minoritarias, una personalización granular que en el entorno cerrado de YOLO resultó ser un obstáculo técnico.

El proceso también estuvo marcado por desafíos significativos en la gestión de recursos computacionales. Debido a la complejidad de las arquitecturas (especialmente los Transformers y CNNs profundas en PyTorch), los tiempos de entrenamiento se extendieron considerablemente. Esto generó problemas de estabilidad, donde algunas sesiones en notebooks colapsaron tras superar las 7 horas de ejecución continua. En varios casos, estos fallos resultaron en la pérdida de checkpoints intermedios del modelo, lográndose rescatar únicamente los registros de métricas (logs) generados hasta el momento del fallo y los mejores pesos. Esta disparidad en el consumo de recursos planteó la siguiente pregunta: ¿Es equitativo comparar un modelo ligero que converge en 2 horas frente a una arquitectura pesada que demanda más de 7 horas y conlleva riesgos de interrupción? Este factor se convirtió en una variable crítica al ponderar no solo la precisión, sino la viabilidad operativa de cada solución.

Para garantizar la integridad de los resultados y evitar sesgos o fugas de datos (data leakage), se estableció un protocolo estricto de evaluación. Todos los modelos fueron entrenados utilizando conjuntos de entrenamiento y validación idénticos, asegurando que ninguno tuviera acceso previo al conjunto de prueba (Test Set). Una vez finalizada la fase de entrenamiento, se seleccionaron los pesos con el mejor rendimiento de validación (best weights) para cada arquitectura. Estos modelos optimizados fueron cargados posteriormente en un entorno aislado, donde se generaron predicciones sobre el conjunto de prueba nunca antes visto. Este método permitió consolidar en un único reporte las métricas de desempeño real y los hiperparámetros óptimos de cada configuración, asegurando una comparativa justa y estandarizada.

Desarrollo de Aplicación

Para la fase final del proyecto, desarrollamos una aplicación interactiva utilizando Streamlit, lo cual hizo que la integración de los modelos fuera un proceso bastante ágil. Desde el punto de vista del código, la implementación fue relativamente sencilla: bastó con importar las librerías estándar y construir algunas funciones de utilidad ("wrappers") para cargar los pesos de los modelos y gestionar las predicciones. Sin embargo, los mayores retos no vinieron de la programación en sí, sino de la gestión del entorno de trabajo. Un obstáculo constante fue la administración de los ambientes virtuales de Python; lograr que las versiones y dependencias fueran consistentes entre las computadoras de los distintos miembros del equipo resultó más complicado de lo esperado, generando conflictos de compatibilidad que frenaron el ritmo en varias ocasiones. Además, debido a los problemas de estabilidad sufridos durante el entrenamiento (donde algunos procesos fallaron tras horas de ejecución), parte de la recolección de datos para la demo tuvo que realizarse de forma manual, reconstruyendo y unificando la información que no se guardó automáticamente tras los cierres inesperados.

Resultados

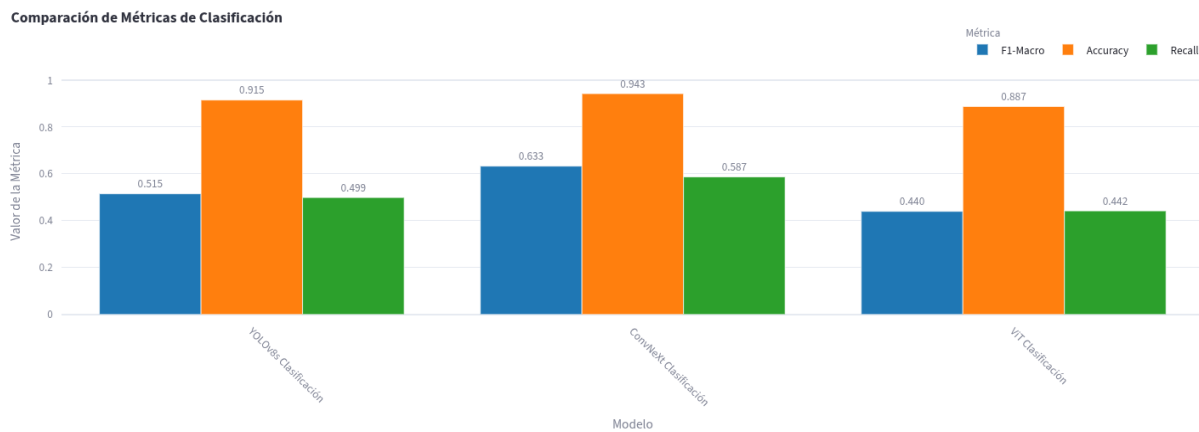
La aplicación de análisis se implementó con éxito, logrando integrar los modelos de detección y clasificación en una interfaz funcional y unificada. El código fuente completo, que incluye tanto los notebooks de entrenamiento como las herramientas de procesamiento, está disponible para su revisión en nuestro repositorio: https://github.com/nel-eleven11/Proyecto2_DataScience. Además, contamos con una demostración grabada en YouTube en el siguiente enlace: <https://youtu.be/oQT28CPXuGI?si=n06vmRoAQyI1Cohu>

Modelos de Clasificación

Las pruebas para estos modelos se realizaron sobre el conjunto de prueba, dónde las imágenes fueron recortadas previamente para asegurarse de tener una evaluación justa. En cuanto a métricas, decidimos utilizar F1-Macro (a comparación de weighted en la Fase 2) para asegurarnos de tomar en cuenta a las clases minoritarias. Adicionalmente, tenemos accuracy y recall como métricas adicionales. Podemos ver que el modelo que más destacó fue ConvNext, seguido de Yolo y por último ViT.

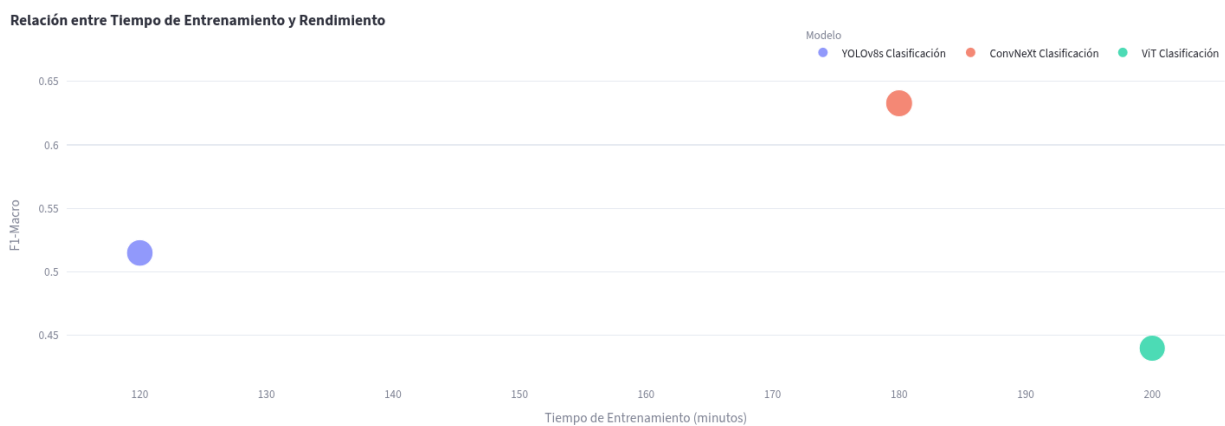
	Modelo	F1-Macro	Accuracy	Recall	Tiempo Entrenamiento (min)
0	YOLOv8s Clasificación	0.5148	0.9150	0.4989	120.0
1	ConvNeXt Clasificación	0.6328	0.9425	0.5868	180.0
2	ViT Clasificación	0.4396	0.8875	0.4418	200.0

Adicionalmente, esto se puede ver de manera visual gracias a la visualización dentro del dashboard implementado dentro del proyecto. Aquí podemos ver que el modelo “ganador” sin tomar en cuenta el costo computacional es ConvNeXt.



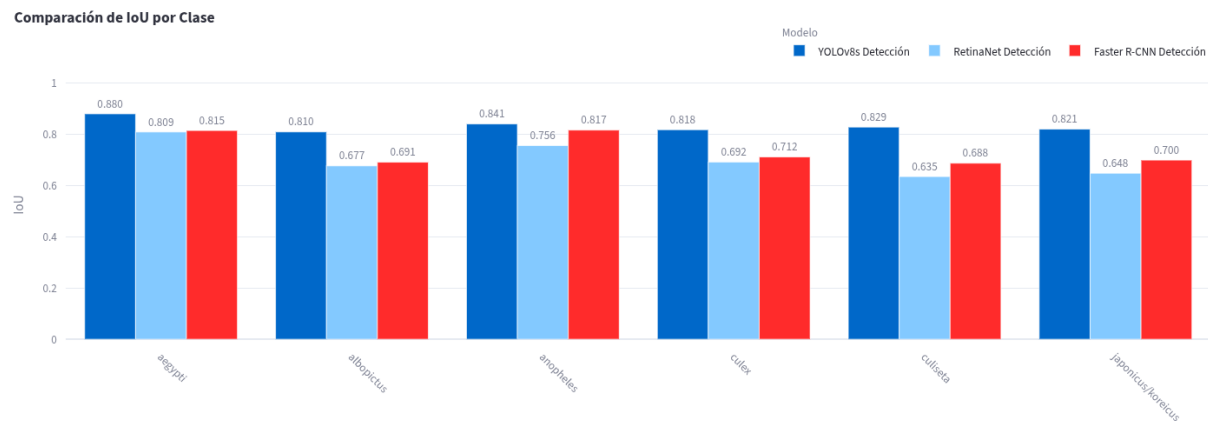
En cuanto a tiempo de entrenamiento y rendimiento, hay dos modelos que realmente valen la pena destacar. YoloV8 tuvo un tiempo de entrenamiento bastante rápido, por lo cual se puede utilizar como un modelo de baseline para este tipo de problemas. Además, logramos identificar a ConvNext bastante por encima de los demás modelos. Esto nos puede indicar que para aplicaciones dónde se cuenta con acceso libre a recursos computacionales puede proveer una mejor solución. Por último, tenemos a ViT que fue el modelo con un entrenamiento más tardío y peor rendimiento de los tres.

Tiempo de Entrenamiento vs Rendimiento

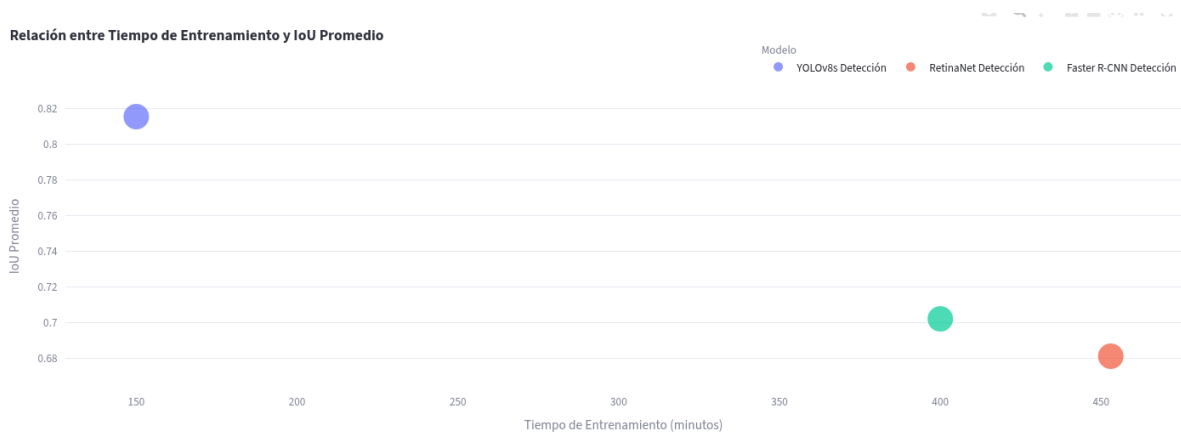


Modelos de Detección

Para los modelos de detección, la métrica principal fue IoU. Esta métrica nos indica el porcentaje de intersección con el área real ocupada por el bounding box del mosquito. Adicionalmente, utilizamos el tiempo de entrenamiento para comparar costo computacional entre los modelos. Iniciando con el IoU por clase, decidimos intentar identificar alguna debilidad de los modelos debido al desbalance de clases. Aquí realmente no logramos encontrar una correlación, tomando en cuenta especialmente que *aegypti* es una clase minoritaria y comparte los IoUs promedios más altos.

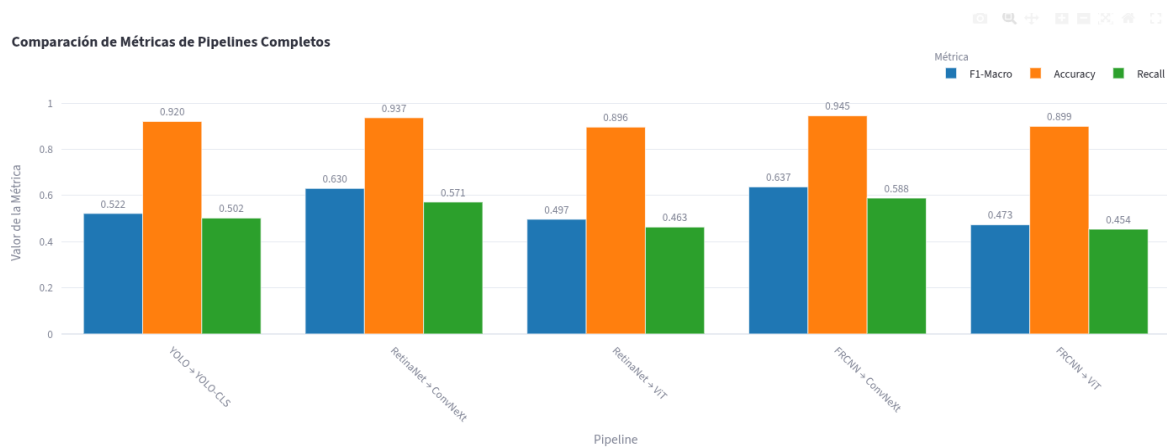


En el caso de los modelos de clasificación, dónde únicamente tenemos una métrica principal, basta una comparación simple de tiempos de entrenamiento e IoU para realizar un análisis comparativo. Aquí prácticamente podemos ver que YoloV8s es el claro ganador. El tiempo de entrenamiento fue significativamente menor en comparación a los demás, que se extendieron entre 6 y 7 horas vs 2.5. Podemos decir que, en el caso de los modelos de clasificación, Yolo sigue siendo una opción muy sólida para establecer un baseline pero no logramos identificar un modelo más robusto para superarlo.



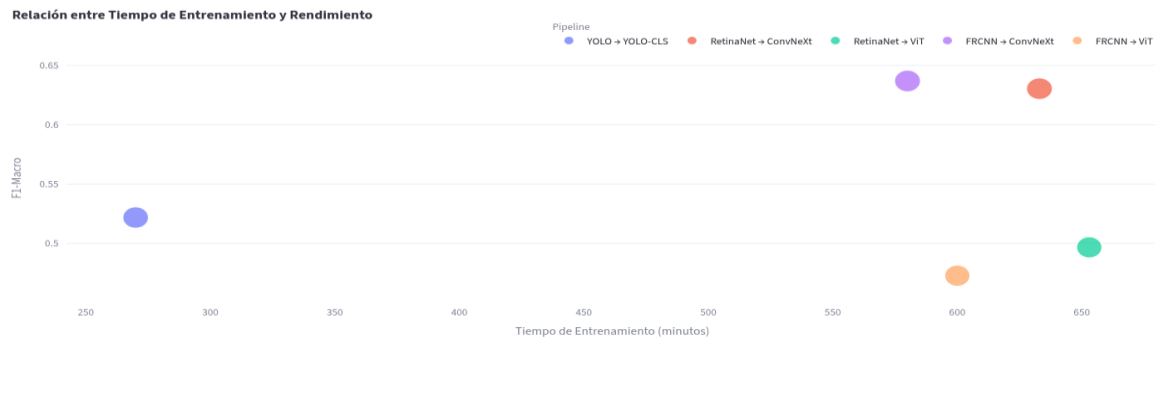
Pipelines Completos

Para los pipelines completos, definimos un pipeline de Yolo “puro”, y luego 4 pipelines de los modelos de PyTorch combinados. El objetivo era comparar un baseline “rápido” desarrollado únicamente utilizando Yolo y su acercamiento “hands off” y compararlo con los modelos más nuevos / recientes de PyTorch. Para la evaluación, principalmente nos centramos en las métricas de clasificación como F1, ya que ese es el objetivo de este proyecto. En comparación estrictamente de métricas, podemos ver que los pipelines completos realmente se encuentran algo parejos en cuanto a accuracy. Sin embargo, la verdadera distinción viene en cuanto al F1 Macro. Los dos pipelines que más resaltan son los que utilizan ConvNext, por lo cual podemos decir que la parte más “importante” dentro de los pipelines realmente es la clasificación. No importa si un modelo logra realizar detecciones precisas si el modelo de clasificación no cumple su objetivo.



Tomando en cuenta el tiempo de entrenamiento, llegamos a hallazgos similares a los incisos anteriores. Los modelos de Yolo proveen un rendimiento bastante alto, considerando su bajo costo computacional y facilidad de entreno. Mientras que los modelos de PyTorch requieren de un tiempo de entrenamiento más alto; y mientras algunos logran tener un rendimiento superior, otros tienen rendimiento aún inferior al baseline.

Tiempo de Entrenamiento vs Rendimiento



Discusión

El desarrollo del proyecto nos llevó por una curva de aprendizaje interesante, pasando de una visión inicial simplista a una arquitectura mucho más compleja. Al principio, la idea de usar modelos "end-to-end" (o "all-in-one") parecía la ruta lógica para resolver detección y clasificación simultáneamente. Sin embargo, los resultados experimentales nos mostraron rápidamente que este enfoque monolítico imponía restricciones severas. Al intentar optimizar tareas tan dispares en una sola red, ninguna terminaba de funcionar al 100%. Esto fue lo que impulsó el cambio hacia una estrategia modular, donde desacoplamos el problema: primero detectar dónde está el mosquito y luego, con una red especializada, decir qué es.

Desafíos Enfrentados

Más allá de la teoría, la implementación práctica trajo consigo varios obstáculos que vale la pena discutir:

- **La Realidad de los Datos:** Nos enfrentamos a imágenes capturadas por ciudadanos en condiciones muy variables, pero el verdadero problema fue el desbalance. Tener dos clases dominantes abarcando el 85% de los datos hizo que nuestros primeros modelos tuvieran un sesgo enorme, ignorando casi por completo a las especies minoritarias.
- **La Trampa de la Abstracción (YOLO vs. PyTorch):** YOLOv8 ("Fast & AIO") nos permitía iterar rapidísimo gracias a su abstracción, pero esa misma "caja negra" se convirtió en un obstáculo cuando quisimos atacar el desbalance de clases. No podíamos inyectar fácilmente un muestreo personalizado. Por otro lado, PyTorch ("Build it on your own") nos dio el control total para implementar el WeightedRandomSampler, pero a costa de escribir

todo el boilerplate manual, lo que aumentó la complejidad del código y la curva de aprendizaje.

- **Estabilidad y Cómputo:** Entrenar modelos pesados como ViT o CNNs profundas en PyTorch fue una prueba de paciencia. Tuvimos sesiones de entrenamiento que superaban las 7 horas y, en varios casos, los *notebooks* colapsaban antes de terminar, perdiendo *checkpoints* y obligándonos a rescatar métricas de los logs de forma manual . Esto nos planteó un dilema: ¿es justo comparar un modelo que tarda 7 horas y es inestable con uno que termina en 2 horas sin problemas?.
- **Conflictos de Dependencias:** Un reto no técnico, pero que impactó el ritmo de trabajo, fue la gestión de entornos. Lograr que las versiones de Python y las librerías fueran consistentes entre las computadoras de todos los miembros del grupo para correr la aplicación de Streamlit fue más complicado de lo esperado, generando conflictos que retrasaron la integración .

Decisiones de Diseño

- **Pipeline de Preprocesamiento y Aumento:** Se optó por estandarizar la entrada utilizando la librería Albumentations, definiendo una resolución uniforme de 320x320 píxeles y aplicando normalización estadística basada en ImageNet.
- **Estrategia de Muestreo (WeightedRandomSampler):** Una de las decisiones más determinantes fue la implementación de un WeightedRandomSampler en los modelos de PyTorch. Dado que dos clases representaban el 85% de los datos, un entrenamiento estándar habría ignorado a las especies minoritarias. Este sampler fuerza al modelo a ver una distribución artificialmente equilibrada en cada batch, una técnica que resultó imposible de replicar con la misma granularidad en la "caja negra" de YOLOv8.
- **Desacople de Modelos:** La decisión de separar el detector del clasificador permitió mezclar lo mejor de dos mundos. Pudimos utilizar detectores rápidos (como YOLOv8 o RetinaNet) cuya única misión es recortar el objeto, y alimentar esos recortes a redes profundas especializadas en características finas (como ConvNeXt) para la clasificación.
- **Métricas de Evaluación:** Las métricas de evaluación utilizadas para los modelos principalmente fueron F1-Macro y IoU. La métrica de F1 provee un balance entre accuracy y recall, mientras que el uso de macro por encima de weighted nos permite no “hacer de

menos” a las clases minoritarias. Por otro lado, IoU es el estándar para cualquier modelo de detección. También consideramos el costo computacional de cada modelo como métrica clave, ya que esto nos da más detalles sobre diferentes casos de uso para cada modelo.

Análisis de Resultados y Lecciones Aprendidas

El desarrollo de este proyecto nos dio un "golpe de realidad" respecto a lo que dicen los papers versus lo que pasa cuando se trabaja con datos reales y desbalanceados. A continuación, detallamos los puntos clave que aprendimos tras pelear con los modelos:

- **No siempre lo más nuevo es mejor (El caso de ViT)** Aunque hoy en día todo el mundo habla de Transformers, para nuestro caso específico, el Vision Transformer (ViT) fue el gran perdedor.
 - Nos dimos cuenta de que aplicar la arquitectura más "moderna" no garantiza nada si no tienes millones de datos.
 - ViT fue el más lento de entrenar (más de 200 minutos) y tuvo un rendimiento inferior (F1 ~0.88) comparado con ConvNeXt (>0.94), lo que confirma que las redes convolucionales modernas siguen siendo las reinas en datasets de tamaño medio.
- **Divide y vencerás: La ventaja del enfoque modular** Al principio, queríamos que un solo modelo hiciera todo ("end-to-end"), pero vimos que el que mucho abarca, poco aprieta.
 - Modelos ligeros como SSD300 fallaron estrepitosamente, con un Recall de apenas 1.9%, porque no lograban aprender ambas tareas a la vez.
 - La solución fue desacoplar: usar un especialista rápido para encontrar el mosquito (detección) y otro robusto para decir qué es (clasificación). El combo ganador resultó ser mezclar frameworks: la velocidad de YOLO para detectar y la precisión de ConvNeXt para clasificar.
- **El detector no discrimina (y eso es bueno)** Viendo las gráficas de IoU, notamos algo curioso: al modelo no le importa si la especie es rara o común para encontrarla.
 - El IoU se mantuvo alto y constante incluso para las clases minoritarias.
 - Esto nos dice que la morfología básica de "un mosquito" (patas, alas, cuerpo) es fácil de aprender para el modelo, independientemente de la especie. Esto valida nuestra decisión de usar un detector genérico al principio del pipeline.
- **Eficiencia: ¿Vale la pena esperar 7 horas?** El tiempo de cómputo fue una variable crítica. Tuvimos sesiones en PyTorch que colapsaban después de 7 horas, lo cual no es sostenible.
 - En contraste, YOLOv8s logró un rendimiento muy sólido en detección en solo 2.5 horas.
 - La lección aquí es simple: para una aplicación en producción (como la que hicimos en Streamlit), no tiene sentido gastar recursos excesivos en la detección si un modelo ligero como YOLO ya te da un IoU arriba de 0.8 de forma mucho más estable.

Conclusiones

La ejecución de este proyecto ha permitido validar empíricamente la eficacia de distintas arquitecturas de aprendizaje profundo aplicadas a la detección. A partir del análisis de las métricas de rendimiento y los costos computacionales, se establecen las siguientes conclusiones:

- **Superioridad del Enfoque Modular:** La evidencia experimental demostró que desacoplar las tareas de localización y clasificación resulta en un desempeño superior al de arquitecturas monolíticas "end-to-end". Mientras que modelos de una sola etapa como SSD300 fracasaron en la convergencia, obteniendo un Recall crítico del 1.9%, la implementación de un pipeline híbrido permitió optimizar cada etapa por separado, maximizando la capacidad de extracción de características.
- **Selección del Modelo de Clasificación (ConvNeXt vs. ViT):** Contrario a las tendencias actuales del estado del arte, la arquitectura ConvNeXt se consolidó como el modelo más robusto para este dominio, superando consistentemente a Vision Transformer (ViT). Los resultados indican que ViT requiere un volumen de datos masivo para generalizar eficazmente; con el dataset actual, ViT presentó el mayor costo computacional (>200 minutos) y el desempeño más bajo en métricas clave como F1-Macro. Por tanto, ConvNeXt se establece como el estándar de calidad para conjuntos de datos de magnitud media al costo de un mayor costo computacional vs Yolo. Si tuviéramos que recomendar modelos, elegiríamos Yolo para obtener un "baseline" y poder experimentar con diferentes modelos más pesados como ConvNeXt.
- **Eficiencia en la Detección (YOLOv8):** Para la etapa de localización, YOLOv8s demostró ser la opción más eficiente, ofreciendo un equilibrio óptimo entre precisión y velocidad. A diferencia de las implementaciones en PyTorch (Faster R-CNN y RetinaNet), que demandaron tiempos de entrenamiento extensos (>6 horas) y presentaron problemas de estabilidad, YOLO completó el entrenamiento en aproximadamente 2.5 horas manteniendo niveles competitivos de IoU. Esto valida su selección como el modelo base para aplicaciones que requieren inferencia ágil.
- **Impacto del Framework en la Gestión del Desbalance:** La elección del entorno de desarrollo fue determinante para mitigar el sesgo hacia las clases mayoritarias, que representaban el 85% de los datos. Aunque el ecosistema de YOLO ("Caja Negra") facilitó la iteración rápida, su abstracción impidió la implementación efectiva de estrategias de remuestreo. La flexibilidad de PyTorch, a pesar de su mayor complejidad de implementación, fue indispensable para integrar el WeightedRandomSampler, garantizando que el modelo aprendiera características de especies minoritarias como *Aedes aegypti*.

Recomendaciones

Basado en los hallazgos técnicos y las limitaciones identificadas durante el desarrollo del proyecto, se proponen las siguientes recomendaciones para trabajos futuros y equipos que aborden problemas similares:

- **Establecimiento de Líneas Base (Baselines) y Prueba de Concepto:** Antes de invertir recursos en el entrenamiento de arquitecturas profundas y costosas, es fundamental establecer una línea base de rendimiento utilizando modelos ligeros y de rápida convergencia (como YOLO). Esto permite validar la integridad del dataset y la viabilidad del problema con un costo computacional mínimo, sirviendo como punto de referencia para justificar o descartar la necesidad de migrar hacia modelos más pesados según el retorno de inversión en precisión.
- **Priorizar Arquitecturas Convolucionales en Datasets Limitados:** Para proyectos con datasets inferiores a las decenas de miles de imágenes, se recomienda evitar arquitecturas basadas puramente en Attention (como ViT). Se sugiere optar por redes convolucionales modernas (como ConvNeXt o EfficientNet), las cuales poseen un sesgo inductivo más adecuado para aprender patrones espaciales con menos datos.
- **Flexibilidad sobre Abstracción en Datos Desbalanceados:** En escenarios con alto desbalance de clases, se recomienda utilizar frameworks que permitan un control granular sobre el bucle de entrenamiento (como PyTorch o TensorFlow nativo) en lugar de herramientas de alto nivel. Esto es crucial para inyectar técnicas de muestreo personalizado o funciones de pérdida específicas (e.g., Focal Loss) que penalicen el sesgo hacia clases dominantes.

Referencias

- Pan American Health Organization (PAHO). (2025). PAHO warns of increased risk of dengue outbreaks due to circulation of DENV-3 in the Americas. Extraído de <https://www.paho.org/en/news/10-2-2025-paho-warns-increased-risk-dengue-outbreaks-due-circulation-denv-3-americas>
- Roiz, D., Pontifes, P.A., Jourdain, F., et al. (2024). The rising global economic costs of invasive Aedes mosquitoes and Aedes-borne diseases. Science of The Total Environment, 934, 173054.

- World Health Organization (WHO). (2024). Vector-borne diseases. Extraído de <https://www.who.int/news-room/fact-sheets/detail/vector-borne-diseases>
- Zhang, W.X., Zhao, T.Y., Wang, C.C., et al. (2025). Assessing the global dengue burden: Incidence, mortality, and disability trends over three decades. PLoS Neglected Tropical Diseases, 19(3), e0012932.
- Carion, N., et al. (2020). End-to-End Object Detection with Transformers (DETR). ECCV.
- Lin, T.-Y., et al. (2017). Focal Loss for Dense Object Detection. ICCV.
- Liu, W., et al. (2016). SSD: Single Shot MultiBox Detector. ECCV.
- Ren, S., et al. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS.
- Tan, M., et al. (2020). EfficientDet: Scalable and Efficient Object Detection. CVPR.
- Ultralytics. (2023). YOLOv8: Real-Time Object Detection. Retrieved from GitHub.